

Enterprise Challenge: Distributed Microservices Performance Testing

Challenge Overview

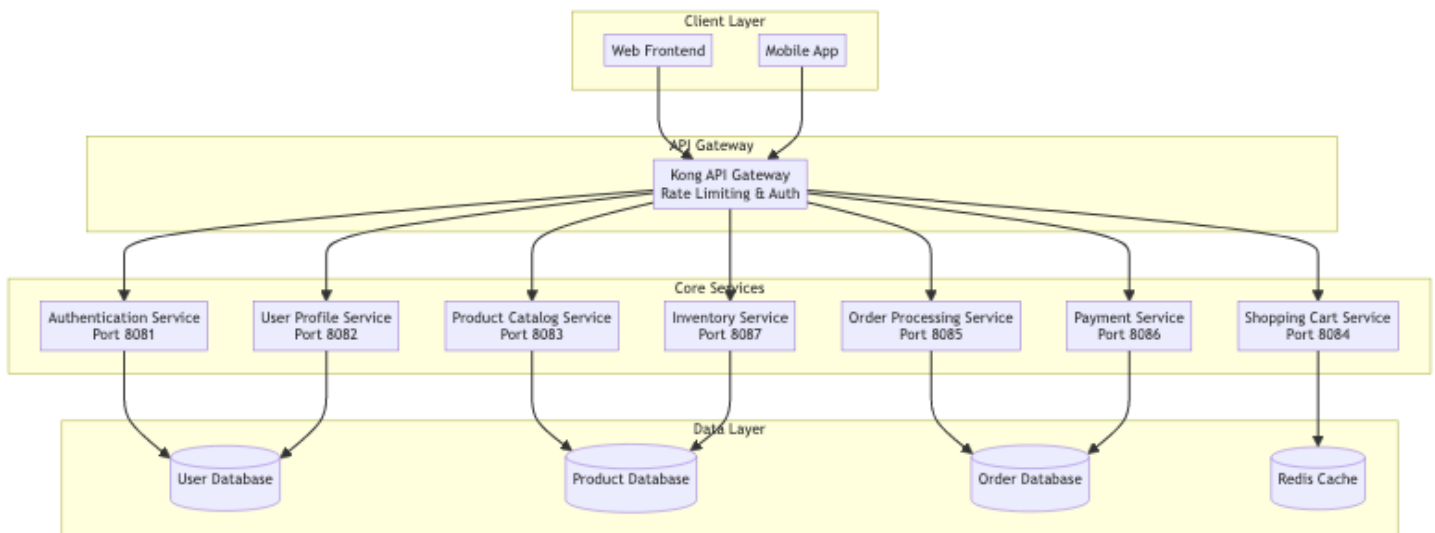
Mission Statement

Design and implement a comprehensive JMeter test plan for a microservices-based e-commerce platform using advanced controllers, distributed execution, and professional reporting standards.

Business Context

ShopTech Solutions is launching a new e-commerce platform built on microservices architecture. The system must handle Black Friday traffic loads while maintaining excellent user experience. Your performance testing will determine production readiness.

System Architecture



Technical Specifications

Service Endpoints and Contracts

Authentication Service (Port 8081)

```
endpoints:
- path: /api/v1/auth/login
  method: POST
  payload: |
    {
      "username": "string",
      "password": "string"
    }
  response: |
    {
      "token": "jwt_token_string",
      "expires": 3600,
      "user_id": "uuid",
      "role": "standard|premium|admin"
    }
  sla:
    response_time_p95: 200ms
    error_rate: <0.1%

- path: /api/v1/auth/validate
```

```

method: GET
headers:
  Authorization: "Bearer {token}"
response: |
{
  "valid": true,
  "user_id": "uuid",
  "expires_in": 3421
}
sla:
  response_time_p95: 100ms
  error_rate: <0.1%

```

User Profile Service (Port 8082)

```

endpoints:
- path: /api/v1/users/{user_id}/profile
  method: GET
  headers:
    Authorization: "Bearer {token}"
    X-Correlation-ID: "uuid"
  response: |
    {
      "user_id": "uuid",
      "username": "string",
      "email": "string",
      "profile": {
        "first_name": "string",
        "last_name": "string",
        "preferences": {}
      }
    }
  sla:
    response_time_p95: 150ms
    error_rate: <0.2%

- path: /api/v1/users/{user_id}/preferences
  method: PUT
  payload: |
    {
      "language": "en",
      "currency": "USD",
      "notifications": true
    }
  sla:
    response_time_p95: 300ms
    error_rate: <0.1%

```

Product Catalog Service (Port 8083)

```

endpoints:
- path: /api/v1/products
  method: GET
  parameters:
    - category: "string (optional)"
    - search: "string (optional)"
    - page: "integer (default: 1)"
    - limit: "integer (default: 20, max: 100)"
  response: |
    {
      "products": [

```

```

    {
      "product_id": "uuid",
      "name": "string",
      "description": "string",
      "price": 29.99,
      "category": "string",
      "inventory_count": 150,
      "images": ["url1", "url2"]
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 1250,
    "has_next": true
  }
}
sla:
  response_time_p95: 300ms
  error_rate: <0.1%

- path: /api/v1/products/{product_id}
  method: GET
  response: |
    {
      "product_id": "uuid",
      "name": "string",
      "description": "string",
      "price": 29.99,
      "category": "string",
      "inventory_count": 150,
      "images": ["url1", "url2"],
      "reviews": {
        "average_rating": 4.2,
        "review_count": 89
      }
    }
  sla:
    response_time_p95: 200ms
    error_rate: <0.1%

```

Shopping Cart Service (Port 8084)

```

endpoints:
- path: /api/v1/cart
  method: POST
  headers:
    Authorization: "Bearer {token}"
    X-Correlation-ID: "uuid"
  payload: |
    {
      "user_id": "uuid",
      "session_id": "uuid"
    }
  response: |
    {
      "cart_id": "uuid",
      "user_id": "uuid",
      "items": [],
      "total": 0.00,

```

```

    "created_at": "2024-01-15T10:00:00Z"
  }
  sla:
    response_time_p95: 150ms
    error_rate: <0.1%

- path: /api/v1/cart/{cart_id}/items
  method: POST
  payload: |
    {
      "product_id": "uuid",
      "quantity": 2,
      "price_per_item": 29.99
    }
  response: |
    {
      "cart_id": "uuid",
      "item_id": "uuid",
      "product_id": "uuid",
      "quantity": 2,
      "price_per_item": 29.99,
      "subtotal": 59.98
    }
  sla:
    response_time_p95: 200ms
    error_rate: <0.2%

- path: /api/v1/cart/{cart_id}
  method: GET
  response: |
    {
      "cart_id": "uuid",
      "user_id": "uuid",
      "items": [
        {
          "item_id": "uuid",
          "product_id": "uuid",
          "product_name": "string",
          "quantity": 2,
          "price_per_item": 29.99,
          "subtotal": 59.98
        }
      ],
      "total": 59.98,
      "item_count": 2
    }
  sla:
    response_time_p95: 150ms
    error_rate: <0.1%

```

Order Processing Service (Port 8085)

endpoints:

```

- path: /api/v1/orders
  method: POST
  headers:
    Authorization: "Bearer {token}"
    X-Correlation-ID: "uuid"
    Idempotency-Key: "uuid"
  payload: |

```

```

{
  "cart_id": "uuid",
  "shipping_address": {
    "street": "123 Main St",
    "city": "Anytown",
    "state": "ST",
    "zip": "12345",
    "country": "US"
  },
  "payment_method_id": "uuid"
}
response: |
{
  "order_id": "uuid",
  "user_id": "uuid",
  "status": "pending_payment",
  "total": 59.98,
  "items": [...],
  "created_at": "2024-01-15T10:00:00Z"
}
sla:
  response_time_p95: 500ms
  error_rate: <0.1%

- path: /api/v1/orders/{order_id}/status
method: GET
response: |
{
  "order_id": "uuid",
  "status": "pending_payment|paid|processing|shipped|delivered|cancelled",
  "tracking_number": "string (if shipped)",
  "estimated_delivery": "2024-01-18T00:00:00Z"
}
sla:
  response_time_p95: 100ms
  error_rate: <0.1%

```

Payment Service (Port 8086)

```

endpoints:
- path: /api/v1/payments
method: POST
headers:
  Authorization: "Bearer {token}"
  X-Correlation-ID: "uuid"
  Idempotency-Key: "uuid"
payload: |
{
  "order_id": "uuid",
  "amount": 59.98,
  "currency": "USD",
  "payment_method": {
    "type": "credit_card",
    "card_token": "secure_token_123"
  }
}
response: |
{
  "payment_id": "uuid",
  "order_id": "uuid",

```

```

    "status": "pending|completed|failed",
    "amount": 59.98,
    "currency": "USD",
    "processed_at": "2024-01-15T10:00:00Z",
    "transaction_id": "txn_123456789"
  }
  sla:
    response_time_p95: 1000ms # Stricter for payment processing
    error_rate: <0.05% # Critical service

- path: /api/v1/payments/{payment_id}/status
  method: GET
  response: |
    {
      "payment_id": "uuid",
      "status": "pending|completed|failed|refunded",
      "failure_reason": "string (if failed)"
    }
  sla:
    response_time_p95: 150ms
    error_rate: <0.1%

```

Non-Functional Requirements

Performance SLAs

Service	P95 Response Time	P99 Response Time	Error Rate	Throughput
Authentication	200ms	500ms	<0.1%	1000 RPS
User Profile	150ms	400ms	<0.2%	800 RPS
Product Catalog	300ms	800ms	<0.1%	2000 RPS
Shopping Cart	200ms	500ms	<0.2%	1500 RPS
Order Processing	500ms	1200ms	<0.1%	500 RPS
Payment Processing	1000ms	2000ms	<0.05%	200 RPS

Load Profile Requirements

- **Peak Concurrent Users:** 5000
- **Peak RPS:** 6000 across all services
- **Test Duration:** 30 minutes sustained load
- **Ramp-up Pattern:** Gradual 10-minute ramp to peak load
- **Geographic Distribution:** 60% US, 25% EU, 15% APAC

Data Constraints and Requirements

Test Data Specifications

- **User Accounts:** 10,000 unique user credentials
- **Product Catalog:** 50,000 products across 20 categories
- **Geographic Regions:** US, EU, APAC routing simulation
- **User Types:** 70% standard, 25% premium, 5% admin
- **Payment Methods:** Credit card tokens (test mode)

Rate Limits and Quotas

- **Authentication:** 100 requests/minute per IP
- **Product Search:** 1000 requests/minute per authenticated user
- **Cart Operations:** 200 requests/minute per user

- **Order Creation:** 10 orders/minute per user
- **Payment Processing:** 5 payments/minute per user

Data Consistency Requirements

- **Eventual Consistency:** Cart-to-inventory synchronization within 5 seconds
- **Strong Consistency:** Payment-to-order status updates immediate
- **Cache TTL:** Product data cached for 300 seconds
- **Session Duration:** User sessions valid for 2 hours

Challenge Requirements

Technical Deliverables

1. JMeter Test Plan Architecture (25 points) Requirements:

- ☐ **Advanced Controllers Implementation**
 - If Controller: User role-based access control
 - Loop Controller: Retry logic with exponential backoff
 - Module Controller: Reusable service interaction patterns
 - Switch Controller: Geographic region routing simulation
- ☐ **Modular Design Patterns**
 - Test Fragments for each service interaction
 - Reusable authentication flow
 - Common header management
 - Error handling patterns
- ☐ **Property-Driven Configuration**
 - Environment-specific property files (dev/staging/prod)
 - Externalized all service endpoints and credentials
 - Load pattern configuration via properties
 - Feature flag simulation via properties

Evaluation Criteria:

- Controller usage appropriateness and complexity
- Code reusability and maintainability
- Configuration management best practices
- Architecture scalability and extensibility

2. Functional Correctness (25 points) Requirements: - [] **Complete User Journey Implementation** - User authentication and session management - Product browsing with search and pagination - Shopping cart operations (add/update/remove items) - Order creation and payment processing - Order status tracking

- ☐ **Service Integration Validation**
 - Proper correlation ID propagation
 - Header management across service calls
 - Idempotency key implementation for critical operations
 - Response data extraction and reuse
- ☐ **Error Handling and Edge Cases**
 - Network timeout scenarios
 - Service unavailability simulation
 - Invalid authentication token handling
 - Payment failure scenarios

Evaluation Criteria:

- End-to-end workflow completeness
- Data correlation accuracy
- Error scenario coverage
- Service contract compliance

3. Scalability and Distributed Execution (20 points) Requirements: - [] Distributed Testing Implementation - Master/remote configuration for multi-node execution - Property distribution to remote agents - Result aggregation and reporting - Remote agent coordination and synchronization

- ☐ **Load Generation Strategy**
 - Realistic user behavior simulation
 - Geographic distribution modeling
 - User type percentage allocation (70% standard, 25% premium, 5% admin)
 - Traffic pattern variation (peak/off-peak simulation)
- ☐ **Resource Optimization**
 - Efficient listener configuration on remote agents
 - Memory and CPU optimization
 - Network bandwidth consideration
 - Backend monitoring integration

Evaluation Criteria:

- Distributed execution functionality
- Load pattern realism and complexity
- Resource utilization efficiency
- Monitoring and observability implementation

4. Analysis and Professional Reporting (20 points) Requirements:

- ☐ **Performance Analysis**
 - Statistical analysis using percentiles (P50, P95, P99)
 - Bottleneck identification using systematic methods
 - Capacity planning calculations with safety factors
 - SLA compliance validation
- ☐ **Executive Summary**
 - Business impact assessment
 - Risk evaluation and recommendations
 - Investment requirements for scaling
 - Timeline for production readiness
- ☐ **Technical Deep Dive**
 - Service-by-service performance breakdown
 - Correlation analysis between services
 - Error pattern analysis and root cause investigation
 - Infrastructure sizing recommendations

Evaluation Criteria:

- Analysis depth and accuracy
- Business communication effectiveness
- Technical insight quality
- Actionable recommendations with priorities

5. Professional Hygiene and Documentation (10 points) Requirements: - [] Repository Structure

```
enterprise-challenge/
├── README.md
├── test-plans/
│   ├── main-test-plan.jmx
│   └── test-fragments/
├── config/
│   ├── dev.properties
│   ├── staging.properties
│   └── production.properties
├── data/
│   ├── users.csv
│   ├── products.csv
│   └── regions.csv
```



```

├── scripts/
│   ├── run-distributed.sh
│   ├── setup-remotes.sh
│   └── generate-report.sh
└── reports/
    ├── executive-summary.md
    ├── technical-analysis.md
    └── html-dashboard/

```

☐ Documentation Quality

- Clear setup and execution instructions
- Architecture decision documentation
- Troubleshooting guide
- Performance baseline documentation

☐ Code Quality

- Descriptive naming conventions
- Inline comments for complex logic
- Version control best practices
- Configuration management

Evaluation Criteria:

- Repository organization and structure
- Documentation completeness and clarity
- Code readability and maintainability
- Professional development practices

Starter Materials

JMX Skeleton Template

```

<?xml version="1.0" encoding="UTF-8"?>
<jmeterTestPlan version="1.2">
  <hashTree>
    <TestPlan testname="ShopTech E-commerce Performance Test" enabled="true">
      <stringProp name="TestPlan.comments">Enterprise microservices performance validation</stringProp>
      <boolProp name="TestPlan.functional_mode">>false</boolProp>
      <boolProp name="TestPlan.serialize_threadgroups">>false</boolProp>
      <elementProp name="TestPlan.arguments" elementType="Arguments">
        <collectionProp name="Arguments.arguments">
          <elementProp name="" elementType="Argument">
            <stringProp name="Argument.name">test.version</stringProp>
            <stringProp name="Argument.value">1.0</stringProp>
          </elementProp>
        </collectionProp>
      </elementProp>
      <stringProp name="TestPlan.user_define_classpath"></stringProp>
    </TestPlan>
  </hashTree>

  <!-- Properties and Configuration -->
  <Arguments testname="Test Configuration" enabled="true">
    <collectionProp name="Arguments.arguments">
      <elementProp name="base.url" elementType="Argument">
        <stringProp name="Argument.value">${__P(base.url,http://localhost)}</stringProp>
      </elementProp>
      <elementProp name="auth.service.port" elementType="Argument">
        <stringProp name="Argument.value">${__P(auth.service.port,8081)}</stringProp>
      </elementProp>
      <elementProp name="user.service.port" elementType="Argument">
        <stringProp name="Argument.value">${__P(user.service.port,8082)}</stringProp>
      </elementProp>
    </collectionProp>
  </Arguments>
</jmeterTestPlan>

```

```

        </elementProp>
        <!-- Add remaining service ports -->
    </collectionProp>
</Arguments>
<hashTree/>

<!-- CSV Data Configuration -->
<CSVDataSet testname="User Data" enabled="true">
    <stringProp name="delimiter">,</stringProp>
    <stringProp name="fileEncoding">UTF-8</stringProp>
    <stringProp name="filename">${__P(test.data.path,./data)}/users.csv</stringProp>
    <boolProp name="ignoreFirstLine">true</boolProp>
    <boolProp name="quotedData">false</boolProp>
    <boolProp name="recycle">true</boolProp>
    <stringProp name="shareMode">shareMode.all</stringProp>
    <boolProp name="stopThread">false</boolProp>
    <stringProp name="variableNames">username,password,email,userType,region</stringProp>
</CSVDataSet>
<hashTree/>

<!-- Main User Journey Thread Group -->
<ThreadGroup testname="E-commerce User Journey" enabled="true">
    <stringProp name="ThreadGroup.on_sample_error">continue</stringProp>
    <elementProp name="ThreadGroup.main_controller" elementType="LoopController">
        <boolProp name="LoopController.continue_forever">false</boolProp>
        <stringProp name="LoopController.loops">1</stringProp>
    </elementProp>
    <stringProp name="ThreadGroup.num_threads">${__P(threads,100)}</stringProp>
    <stringProp name="ThreadGroup.ramp_time">${__P(rampup,300)}</stringProp>
    <longProp name="ThreadGroup.start_time">1578876963000</longProp>
    <longProp name="ThreadGroup.end_time">1578876963000</longProp>
    <boolProp name="ThreadGroup.scheduler">false</boolProp>
    <stringProp name="ThreadGroup.duration"></stringProp>
    <stringProp name="ThreadGroup.delay"></stringProp>
</ThreadGroup>
<hashTree>

<!-- Region-based routing with Switch Controller -->
<SwitchController testname="Geographic Routing" enabled="true">
    <stringProp name="SwitchController.value">${__groovy(
        vars.get("region").equals("US") ? "0" :
        vars.get("region").equals("EU") ? "1" : "2"
    )}</stringProp>
</SwitchController>
<hashTree>

<!-- US Region Configuration -->
<GenericController testname="US Region" enabled="true"/>
<hashTree>
    <Arguments testname="US Configuration" enabled="true">
        <collectionProp name="Arguments.arguments">
            <elementProp name="region.code" elementType="Argument">
                <stringProp name="Argument.value">US</stringProp>
            </elementProp>
            <elementProp name="currency" elementType="Argument">
                <stringProp name="Argument.value">USD</stringProp>
            </elementProp>
        </collectionProp>
    </Arguments>
</hashTree/>

```

```

</hashTree>

<!-- EU Region Configuration -->
<GenericController testname="EU Region" enabled="true"/>
<hashTree>
  <Arguments testname="EU Configuration" enabled="true">
    <collectionProp name="Arguments.arguments">
      <elementProp name="region.code" elementType="Argument">
        <stringProp name="Argument.value">EU</stringProp>
      </elementProp>
      <elementProp name="currency" elementType="Argument">
        <stringProp name="Argument.value">EUR</stringProp>
      </elementProp>
    </collectionProp>
  </Arguments>
</hashTree/>
</hashTree>

<!-- APAC Region Configuration -->
<GenericController testname="APAC Region" enabled="true"/>
<hashTree>
  <Arguments testname="APAC Configuration" enabled="true">
    <collectionProp name="Arguments.arguments">
      <elementProp name="region.code" elementType="Argument">
        <stringProp name="Argument.value">APAC</stringProp>
      </elementProp>
      <elementProp name="currency" elementType="Argument">
        <stringProp name="Argument.value">SGD</stringProp>
      </elementProp>
    </collectionProp>
  </Arguments>
</hashTree/>
</hashTree>

<!-- Authentication Module -->
<ModuleController testname="User Authentication" enabled="true">
  <stringProp name="ModuleController.module">Authentication Flow</stringProp>
</ModuleController>
</hashTree/>

<!-- User type-based flow with If Controller -->
<IfController testname="Premium User Features" enabled="true">
  <stringProp name="IfController.condition">${__groovy(vars.get("userType").equals("premium"))}
</IfController>
<hashTree>
  <ModuleController testname="Premium Features Access" enabled="true">
    <stringProp name="ModuleController.module">Premium Features</stringProp>
  </ModuleController>
</hashTree/>
</hashTree>

<!-- Main Shopping Flow -->
<ModuleController testname="Product Browsing" enabled="true">
  <stringProp name="ModuleController.module">Browse Products</stringProp>
</ModuleController>
</hashTree/>

<!-- Shopping Cart with probability -->

```

```

<ThroughputController testname="Add to Cart (60% probability)" enabled="true">
  <intProp name="ThroughputController.style">1</intProp>
  <boolProp name="ThroughputController.perThread">true</boolProp>
  <intProp name="ThroughputController.maxThroughput">1</intProp>
  <FloatProperty>
    <name>ThroughputController.percentThroughput</name>
    <value>60.0</value>
  </FloatProperty>
</ThroughputController>
<hashTree>
  <ModuleController testname="Shopping Cart Operations" enabled="true">
    <stringProp name="ModuleController.module">Cart Management</stringProp>
  </ModuleController>
  <hashTree/>
</hashTree>

<!-- Checkout Flow with lower probability -->
<ThroughputController testname="Complete Purchase (25% probability)" enabled="true">
  <intProp name="ThroughputController.style">1</intProp>
  <boolProp name="ThroughputController.perThread">true</boolProp>
  <intProp name="ThroughputController.maxThroughput">1</intProp>
  <FloatProperty>
    <name>ThroughputController.percentThroughput</name>
    <value>25.0</value>
  </FloatProperty>
</ThroughputController>
<hashTree>
  <ModuleController testname="Order and Payment" enabled="true">
    <stringProp name="ModuleController.module">Checkout Process</stringProp>
  </ModuleController>
  <hashTree/>
</hashTree>

</hashTree>

<!-- Test Fragment: Authentication Flow -->
<TestFragmentController testname="Authentication Flow" enabled="false"/>
<hashTree>

  <HTTPSamplerProxy testname="Login Request" enabled="true">
    <elementProp name="HTTPsampler.Arguments" elementType="Arguments">
      <collectionProp name="Arguments.arguments"/>
    </elementProp>
    <stringProp name="HTTPSampler.domain">${base.url}</stringProp>
    <stringProp name="HTTPSampler.port">${auth.service.port}</stringProp>
    <stringProp name="HTTPSampler.protocol">http</stringProp>
    <stringProp name="HTTPSampler.contentEncoding">UTF-8</stringProp>
    <stringProp name="HTTPSampler.path">/api/v1/auth/login</stringProp>
    <stringProp name="HTTPSampler.method">POST</stringProp>
    <boolProp name="HTTPSampler.postBodyRaw">true</boolProp>
    <elementProp name="HTTPsampler.Arguments" elementType="Arguments">
      <stringProp name="Argument.value">{
"username": "${username}",
"password": "${password}"
}</stringProp>
    </elementProp>
  </HTTPSamplerProxy>
  <hashTree>

    <HeaderManager testname="Auth Headers" enabled="true">

```

```

    <collectionProp name="HeaderManager.headers">
      <elementProp name="" elementType="Header">
        <stringProp name="Header.name">Content-Type</stringProp>
        <stringProp name="Header.value">application/json</stringProp>
      </elementProp>
      <elementProp name="" elementType="Header">
        <stringProp name="Header.name">X-Correlation-ID</stringProp>
        <stringProp name="Header.value">${__UUID()}</stringProp>
      </elementProp>
    </collectionProp>
  </HeaderManager>
</hashTree/>

<JSONExtractor testname="Extract Auth Token" enabled="true">
  <stringProp name="JSONExtractor.referenceNames">authToken</stringProp>
  <stringProp name="JSONExtractor.jsonPathExpressions">$.token</stringProp>
  <stringProp name="JSONExtractor.defaultValues">AUTH_FAILED</stringProp>
  <stringProp name="JSONExtractor.matchNumbers">1</stringProp>
</JSONExtractor>
</hashTree/>

<JSONExtractor testname="Extract User ID" enabled="true">
  <stringProp name="JSONExtractor.referenceNames">userId</stringProp>
  <stringProp name="JSONExtractor.jsonPathExpressions">$.user_id</stringProp>
  <stringProp name="JSONExtractor.defaultValues">USER_ID_FAILED</stringProp>
  <stringProp name="JSONExtractor.matchNumbers">1</stringProp>
</JSONExtractor>
</hashTree/>

<ResponseAssertion testname="Login Success Assertion" enabled="true">
  <collectionProp name="Assertion.test_strings">
    <stringProp name="200">200</stringProp>
  </collectionProp>
  <stringProp name="Assertion.test_field">Assertion.response_code</stringProp>
  <intProp name="Assertion.test_type">1</intProp>
</ResponseAssertion>
</hashTree/>

</hashTree>

</hashTree>

<!-- Additional Test Fragments would be defined here -->
<!-- Browse Products, Cart Management, Checkout Process, etc. -->

<!-- Results Listener (minimal for distributed execution) -->
<ResultCollector testname="Simple Data Writer" enabled="true">
  <boolProp name="ResultCollector.errorLogging">>false</boolProp>
  <objProp>
    <n>saveConfig</n>
    <value class="SampleSaveConfiguration">
      <time>true</time>
      <latency>true</latency>
      <timestamp>true</timestamp>
      <success>true</success>
      <label>true</label>
      <code>true</code>
      <message>false</message>
      <threadName>false</threadName>
      <dataType>false</dataType>
    </value>
  </objProp>
</ResultCollector>

```

```

        <encoding>false</encoding>
        <assertions>false</assertions>
        <subresults>false</subresults>
        <responseData>false</responseData>
        <samplerData>false</samplerData>
        <xml>false</xml>
        <fieldNames>false</fieldNames>
        <responseHeaders>false</responseHeaders>
        <requestHeaders>false</requestHeaders>
    </value>
</objProp>
<stringProp name="filename">${__P(results.file,results.jtl)}</stringProp>
</ResultCollector>
<hashTree/>

</hashTree>
</hashTree>
</jmeterTestPlan>

```

Properties File Examples

Development Environment (dev.properties)

```

# Development Environment Configuration
test.environment=development
base.url=http://dev-api.shoptech.com
auth.service.port=8081
user.service.port=8082
product.service.port=8083
cart.service.port=8084
order.service.port=8085
payment.service.port=8086

```

```

# Load Configuration
threads=50
rampup=120
duration=300
loops=1

```

```

# Test Data
test.data.path=./data
results.file=dev-results.jtl

```

```

# Feature Flags
feature.premium.enabled=true
feature.new.checkout=false

```

```

# Monitoring
monitoring.enabled=false
influxdb.host=localhost
influxdb.port=8086
influxdb.database=jmeter_dev

```

Staging Environment (staging.properties)

```

# Staging Environment Configuration
test.environment=staging
base.url=https://staging-api.shoptech.com
auth.service.port=443
user.service.port=443
product.service.port=443

```

```

cart.service.port=443
order.service.port=443
payment.service.port=443

# Load Configuration
threads=500
rampup=300
duration=900
loops=1

# Test Data
test.data.path=./data
results.file=staging-results-${__time(YMDHMS)}.jtl

# Feature Flags
feature.premium.enabled=true
feature.new.checkout=true

# Monitoring
monitoring.enabled=true
influxdb.host=monitoring.shoptech.com
influxdb.port=8086
influxdb.database=jmeter_staging

# Rate Limits
rate.limit.auth=100
rate.limit.search=1000
rate.limit.cart=200

```

Production Environment (production.properties)

```

# Production Environment Configuration
test.environment=production
base.url=https://api.shoptech.com
auth.service.port=443
user.service.port=443
product.service.port=443
cart.service.port=443
order.service.port=443
payment.service.port=443

# Load Configuration
threads=5000
rampup=600
duration=1800
loops=1

# Test Data
test.data.path=./data
results.file=production-results-${__time(YMDHMS)}.jtl

# Feature Flags
feature.premium.enabled=true
feature.new.checkout=true

# Monitoring
monitoring.enabled=true
influxdb.host=monitoring.shoptech.com
influxdb.port=8086
influxdb.database=jmeter_production

```

```
# Security
ssl.verification=true
api.timeout.connect=5000
api.timeout.response=30000

# Distributed Testing
remote.hosts=load-gen-1.shoptech.com,load-gen-2.shoptech.com,load-gen-3.shoptech.com
```

Acceptance Criteria

Functional Requirements

- ☐ **Complete User Journey:** All major user flows implemented and working
- ☐ **Service Integration:** Proper correlation and data flow between services
- ☐ **Error Handling:** Graceful handling of service failures and timeouts
- ☐ **Authentication:** Secure token-based authentication throughout journey
- ☐ **Data Validation:** Proper assertions for response validation

Performance Requirements

- ☐ **Load Pattern Adherence:** Realistic user behavior and traffic distribution
- ☐ **SLA Compliance:** All services meet specified performance targets
- ☐ **Error Rate Targets:** Error rates within specified thresholds (<0.1% for most services)
- ☐ **Concurrent User Support:** Successfully simulate 5000+ concurrent users
- ☐ **Response Time Validation:** P95 response times meet service-specific targets

Technical Requirements

- ☐ **Advanced Controllers:** Proper implementation of If/Loop/Module/Switch controllers
- ☐ **Distributed Execution:** Successful multi-node test execution with result aggregation
- ☐ **Property Configuration:** Environment-agnostic test plan with externalized config
- ☐ **Monitoring Integration:** Real-time metrics collection and dashboard generation
- ☐ **Resource Efficiency:** Optimized resource usage on remote agents

Professional Requirements

- ☐ **Executive Communication:** Business-focused performance summary with recommendations
- ☐ **Technical Analysis:** Detailed performance analysis with statistical rigor
- ☐ **Documentation Quality:** Professional-grade documentation and repository organization
- ☐ **Reproducible Execution:** Clear instructions for test setup and execution
- ☐ **Version Control:** Professional Git practices with meaningful commit messages

Evaluation Rubric

Scoring Breakdown (100 Total Points)

Architecture & Design (25 points)

Criteria	Excellent (23-25)	Good (18-22)	Satisfactory (13-17)	Needs Improvement (0-12)
Controller Usage	Advanced controllers used appropriately with complex logic	Controllers used correctly with moderate complexity	Basic controller implementation	Minimal or incorrect controller usage
Modularity	Highly modular design with excellent reusability	Good modular structure with some reusability	Basic modular approach	Monolithic design with poor structure

Criteria	Excellent (23-25)	Good (18-22)	Satisfactory (13-17)	Needs Improvement (0-12)
Configuration	Comprehensive property-driven configuration	Good externalization of key settings	Basic property usage	Hard-coded values throughout

Technical Correctness (25 points)

Criteria	Excellent (23-25)	Good (18-22)	Satisfactory (13-17)	Needs Improvement (0-12)
Functional Accuracy	All services work correctly with proper data flow	Most services working with minor issues	Basic functionality with some gaps	Major functional problems
Correlation & Integration	Perfect data correlation across service chain	Good correlation with minor issues	Basic correlation implementation	Poor or missing correlation
Error Handling	Comprehensive error scenarios and recovery	Good error handling coverage	Basic error handling	Minimal error consideration

Scalability & Distribution (20 points)

Criteria	Excellent (18-20)	Good (14-17)	Satisfactory (10-13)	Needs Improvement (0-9)
Distributed Execution	Flawless multi-node execution with optimization	Good distributed setup with minor issues	Basic distributed functionality	Poor or non-functional distribution
Load Patterns	Realistic, complex user behavior simulation	Good load pattern implementation	Basic load pattern	Simple or unrealistic load
Resource Optimization	Excellent resource usage and monitoring	Good optimization practices	Basic resource management	Poor resource utilization

Analysis & Reporting (20 points)

Criteria	Excellent (18-20)	Good (14-17)	Satisfactory (10-13)	Needs Improvement (0-9)
Statistical Analysis	Sophisticated analysis with business insights	Good statistical approach	Basic statistical analysis	Poor or missing analysis
Business Communication	Executive-ready communication with clear ROI	Good business language and recommendations	Basic business communication	Technical jargon without business context
Technical Depth	Deep technical insights with actionable recommendations	Good technical analysis	Basic technical findings	Superficial technical coverage

Professional Hygiene (10 points)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
Documentation	Comprehensive, professional documentation	Good documentation coverage	Basic documentation	Poor or missing documentation

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Needs Improvement (0-4)
Repository Organization	Excellent structure and organization	Good repository structure	Basic organization	Poor structure
Code Quality	Clean, well-commented, maintainable code	Good code quality practices	Acceptable code quality	Poor code quality

Grading Scale

- **A (90-100 points):** Exceptional work demonstrating mastery of enterprise performance testing
- **B (80-89 points):** Good work meeting most professional standards
- **C (70-79 points):** Satisfactory work meeting basic requirements
- **D (60-69 points):** Below expectations, significant improvement needed
- **F (0-59 points):** Unsatisfactory work not meeting minimum standards

Submission Guidelines

Submission Format

- ☐ **GitHub Repository:** Public repository with complete source code
- ☐ **Repository Name:** {lastname}-{firstname}-enterprise-perf-challenge
- ☐ **Due Date:** Session 5 start time (individual presentations)
- ☐ **Presentation Duration:** 5 minutes per student + 2 minutes Q&A

Required Deliverables Checklist

- ☐ Complete JMeter test plan (.jmx file)
- ☐ Property files for all environments
- ☐ Test data files (CSV format)
- ☐ Distributed execution scripts
- ☐ HTML Dashboard report
- ☐ Executive summary document
- ☐ Technical analysis report
- ☐ README with setup instructions
- ☐ Repository with professional organization

Presentation Requirements

- ☐ **Demo:** Live execution of distributed test (2 minutes)
- ☐ **Results:** Key performance findings presentation (2 minutes)
- ☐ **Architecture:** Brief technical architecture explanation (1 minute)
- ☐ **Q&A:** Answer peer and instructor questions

Late Submission Policy

- **0-24 hours late:** -10 points
- **24-48 hours late:** -20 points
- **48+ hours late:** -50 points
- **No submission:** 0 points

Success Tips and Recommendations

Getting Started Strategy

1. **Start with Architecture:** Design your test structure before implementation

2. **Incremental Development:** Build one service at a time, test thoroughly
3. **Early Distribution Testing:** Set up distributed execution early to avoid last-minute issues
4. **Property-First Approach:** Externalize configuration from the beginning
5. **Continuous Validation:** Test each component as you build it

Common Pitfalls to Avoid

- **Over-Engineering:** Keep it professional but don't add unnecessary complexity
- **Hard-Coded Values:** Use properties for all environment-specific settings
- **Poor Error Handling:** Plan for and test failure scenarios
- **Ignoring Performance:** Monitor resource usage during development
- **Documentation Afterthought:** Document as you develop, not at the end

Technical Excellence Tips

- **Correlation Patterns:** Master data extraction and reuse between services
- **Header Management:** Implement proper correlation ID and tracing headers
- **Retry Logic:** Implement realistic retry patterns with exponential backoff
- **Circuit Breakers:** Simulate service failure and recovery scenarios
- **Monitoring Integration:** Use Backend Listener for real-time performance tracking

Professional Communication

- **Executive Summary:** Focus on business impact and investment recommendations
- **Technical Analysis:** Provide specific, actionable optimization recommendations
- **Data Visualization:** Use charts and graphs to support your narrative
- **Risk Assessment:** Clearly communicate production readiness and risk factors

This enterprise challenge reflects the complexity and expectations of professional performance engineering roles in large-scale distributed systems environments.